IT5403
OPERATING SYSTEMS

Simplicity and Reliability: A Demo of FreeRTOS

Done By

Kabilesh Selvakumar E	2022506078
Yuvaraj K	2022506079
Seshathri A	2022506102
Krishnaa S	2022506104
Abdul Rahman S M	2022506311

DEPARTMENT OF INFORMATION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY,
ANNA UNIVERSITY, CHENNAI – 600 044

May 2024

TABLE OF CONTENTS

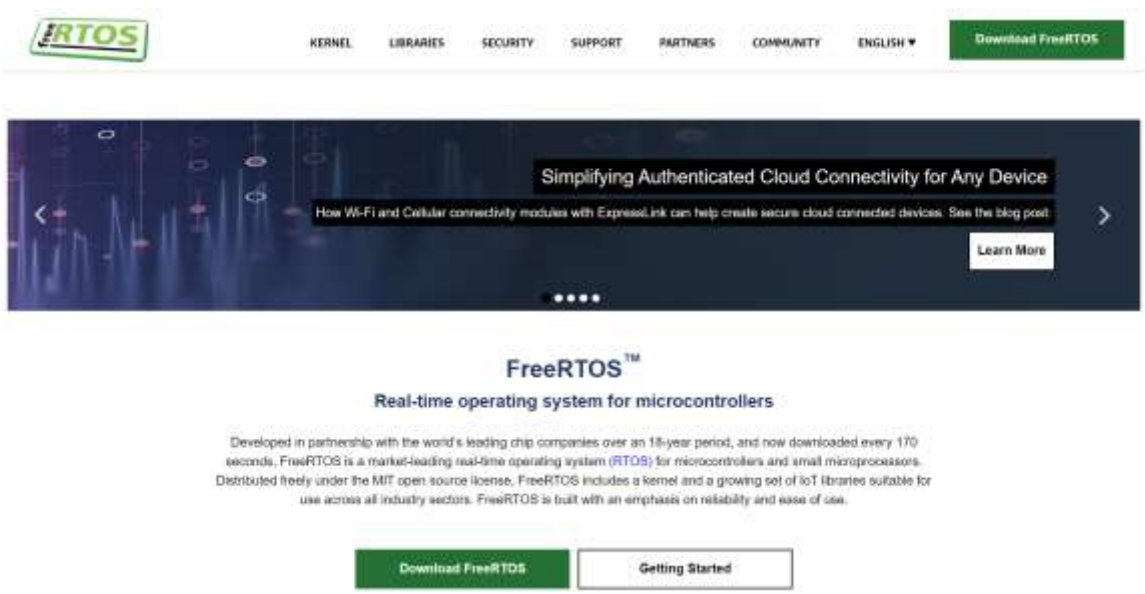
S. No	Title	Page No
1	Installation of FreeRTOS	1
2	Scheduling Algorithms	4
3	Queues and Software Timers	10
4	References	20

Installation of FreeRTOS

FreeRTOS is quite different from the traditional operating systems such as Android, Windows, and MacOS that we encounter in our day-to-day lives. In the simplest of terms, this real-time operating system is nothing a bunch of C files that talk with each other and enable scheduling, thread handling, and other similar tasks for critical operations that need to be executed within the given timeframe. As such, it is not designed to handle complex user interfaces, web applications, and the like. FreeRTOS is a lightweight operating system that focuses on providing a real-time kernel for managing tasks, scheduling, and synchronisation.

The installation of FreeRTOS on Windows is quite simple and straightforward.

1) Go to freertos.org.



2) The *Download FreeRTOS* option is clearly visible. Click on that to navigate to the download options.

3) The first option consists of the FreeRTOS Kernel, the FreeRTOS-Plus libraries, AWS IoT libraries, and several sample projects. This can be downloaded to get a real, in-depth understanding of how FreeRTOS works for different ports and architectures and how we can use them for our microcontroller devices.

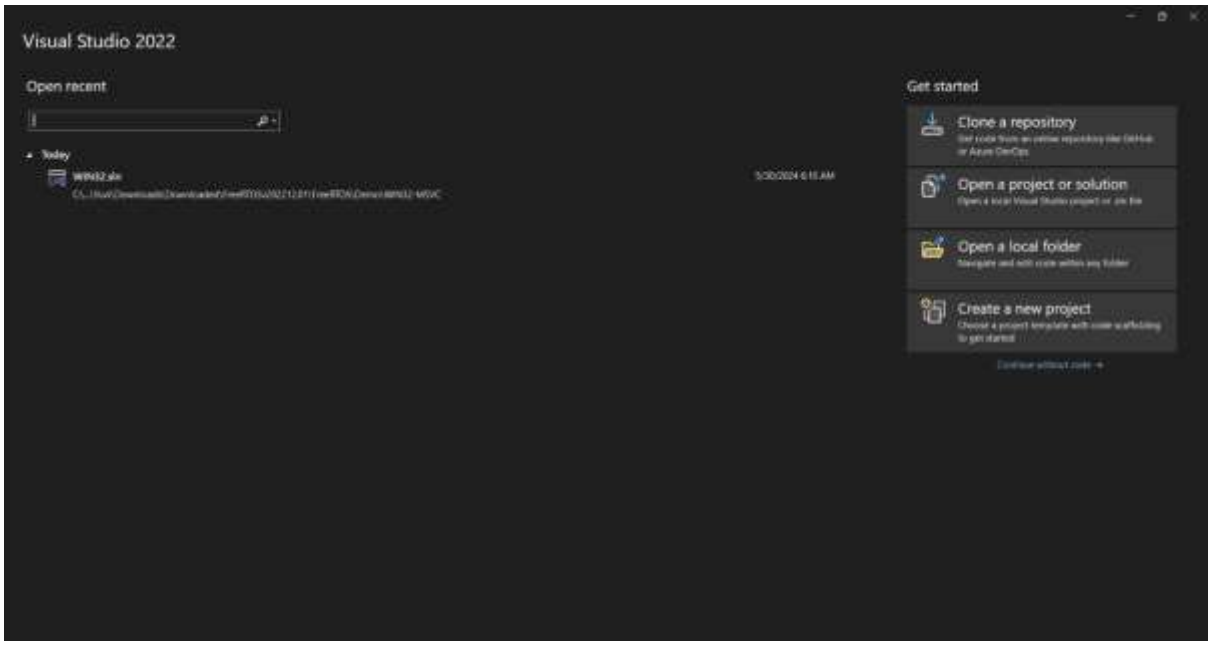
4) Once the required option is selected, the download will begin. It will be a *.zip* file that can be downloaded to the user's desired folder.

5) Once the *.zip* file is downloaded, the contents in it have to be extracted. In some devices, the default Windows-supported extraction tool will be able to complete this task successfully. However, in a few devices, there will be an error along the lines of some file names being lengthier than what is supported. In such cases, a third party extraction software such as Total Commander should come in handy.

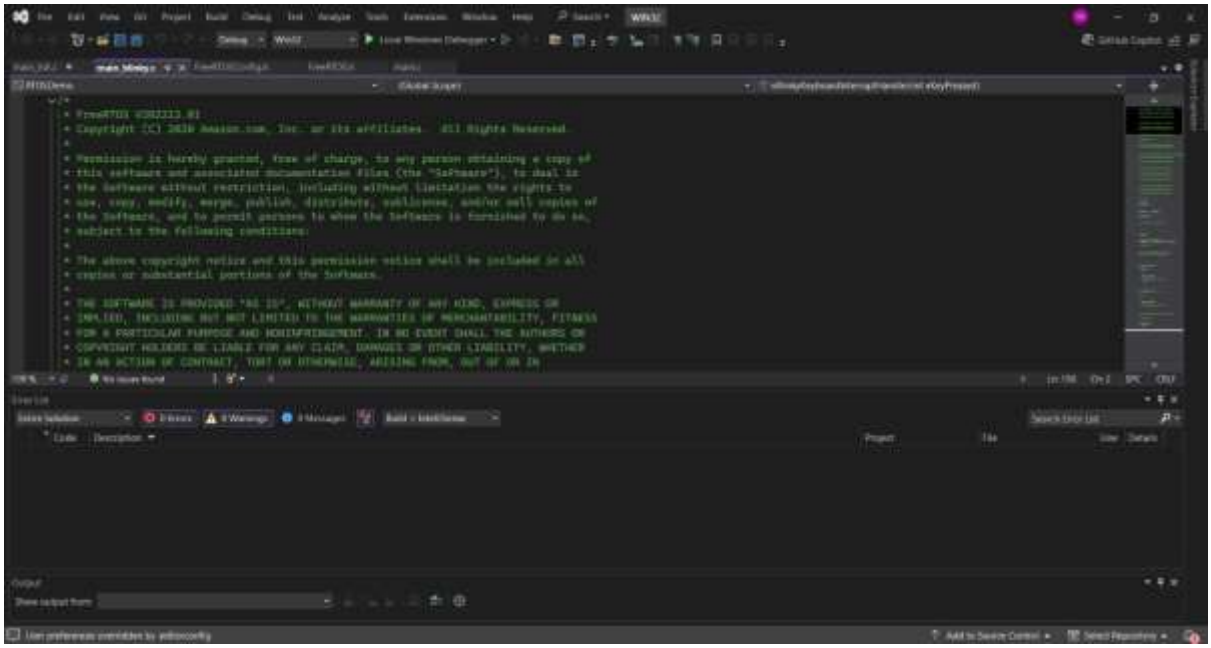
6) It is also necessary to have an IDE to build the FreeRTOS projects. Visual Studio is a popular and well-established option that provides support for the FreeRTOS C code and additional tools that it requires.

7) In Visual Studio, what we need to do is open a *solution* file. Choose the corresponding option, and navigate to the location where FreeRTOS has been extracted.

8) Go to FreeRTOS/Demo/WIN32-MSVC, and in this folder, choose the *win32.sln* file. The project will now be opened. If Visual Studio requests for any additional tools to be installed, do so accordingly.



10) FreeRTOS for the Win32 architecture has successfully been loaded. It is one of the many sample projects that already has many useful demos and instructions that will be helpful to understand the workings of FreeRTOS. Changes and alterations can be made to these files as per the user's desires in order to experiment with the different functionalities supported by FreeRTOS.



Scheduling Algorithms

Scheduling Algorithm	Prioritised	configUSE_PREEMPTION	configUSE_TIME_SLICING
Preemptive with Time Slicing	Yes	1	1
Preemptive without Time Slicing	Yes	1	0
Co-operative	No	0	Any

Altering these macros in *FreeRTOSConfig.h* will enable the different scheduling algorithms.

COMMON SOURCE CODE

```
/* Standard includes. */
#include <stdio.h>

/* FreeRTOS kernel includes. */
#include "FreeRTOS.h"
#include "task.h"

/* Priorities at which the tasks are created. */
#define task1PRIORITY       ( tskIDLE_PRIORITY + 1 )
#define task2PRIORITY       ( tskIDLE_PRIORITY + 1 )

#define mainRESET_TIMER_KEY                ( 'r' )

/*-----------------------------------------------------------*/

/*
 * The tasks as described in the comments at the top of this file.
 */
```

```
static void prvTask1(void* pvParameters);
static void prvTask2(void* pvParameters);

/*-----------------------------------------------------------*/

void main_blinky(void)
{
   printf("\r\nStarting the blinky demo.\r\n\r\n");

   /* Create the two tasks with the same priority. */
   xTaskCreate(prvTask1, "Task1", configMINIMAL_STACK_SIZE, NULL,
task1PRIORITY, NULL);
   xTaskCreate(prvTask2, "Task2", configMINIMAL_STACK_SIZE, NULL,
task2PRIORITY, NULL);

   /* Start the scheduler. */
   vTaskStartScheduler();

   /* If all is well, the scheduler will now be running, and the following
   line will never be reached. If the following line does execute, then
   there was insufficient FreeRTOS heap memory available for the idle and/or
   timer tasks to be created. See the memory management section on the
   FreeRTOS web site for more details. */
   for (;; );
}

/*-----------------------------------------------------------*/

static void prvTask1(void* pvParameters)
{
   (void)pvParameters;

   for (;; )
   {
      /* Task 1 implementation */
      printf("Task 1 running.\r\n");

      /* Cooperatively yield to Task 2 */
      // vTaskDelay(pdMS_TO_TICKS(1000));  // Delay for 1000 ms
      // taskYIELD();
```

```
    }
}

/*-----------------------------------------------------------*/

static void prvTask2(void* pvParameters)
{
    (void)pvParameters;

    for (;; )
    {
        /* Task 2 implementation */
        printf("Task 2 running.\r\n");

        /* Cooperatively yield to Task 1 */
        //vTaskDelay(pdMS_TO_TICKS(1000));  // Delay for 1000 ms
        //taskYIELD();
    }
}

/* Called from prvKeyboardInterruptSimulatorTask(), which is defined in
main.c. */
void vBlinkyKeyboardInterruptHandler(int xKeyPressed)
{
    /* Handle keyboard input. */
    switch (xKeyPressed)
    {
    case mainRESET_TIMER_KEY:
    default:
        break;
    }
}
```

OUTPUT – PRIORITISED PREEMPTIVE WITH TIME SLICING

```
Task 1 running.
Task 1 running.
Task 1 running.
```

OUTPUT – PRIORITISED PREEMPTIVE WITHOUT TIME SLICING

```
Task 1 running.
Task 1 running.
Task 1 running.
```

OUTPUT – COOPERATIVE SCHEDULING

```
Trace started.
The trace will be dumped to the file "Trace.dump" whenever a call to configASSERT()
fails or the 't' key is pressed.
Note that the trace output uses the ring buffer mode, meaning that the output trace
will only be the most recent data able to fit within the trace recorder buffer.

Starting the blinky demo.

Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
Task 2 running.
Task 1 running.
```

Queues and Software Timers

QUEUES

Queues are First in First Out (FIFO) buffers. Data is written to the end/tail of the queue, and removed from the front/head of the queue. Any number of tasks can write to the same queue, and any number of tasks can read from the same queue.

Tasks can be blocked from reading from a queue to ensure that they wait for data to become available in the queue. Similarly, tasks can be blocked from writing to a queue to ensure that there is space available on the queue for data to be written into it.

SOFTWARE TIMERS

Software timers are used to schedule the execution of a function at a set time in the future, or periodically with a fixed frequency. They are optional, and are included by building the source file FreeRTOS/Source/timers.c as a part of the project, plus defining some constants in the FreeRTOSConfig.h header file.

SOURCE CODE

```
*****************************************************************
**************
 *
 * main_blinky() creates one queue, one software timer, and two tasks.  It then
 * starts the scheduler.
 *
 * The Queue Send Task:
 * The queue send task is implemented by the prvQueueSendTask() function in
 * this file.  It uses vTaskDelayUntil() to create a periodic task that sends
 * the value 100 to the queue every 200 milliseconds (please read the notes
 * above regarding the accuracy of timing under Windows).
 *
 * The Queue Send Software Timer:
 * The timer is a one-shot timer that is reset by a key press.  The timer's
 * period is set to two seconds - if the timer expires then its callback
 * function writes the value 200 to the queue.  The callback function is
```

```
 * implemented by prvQueueSendTimerCallback() within this file.
 *
 * The Queue Receive Task:
 * The queue receive task is implemented by the prvQueueReceiveTask()
function
 * in this file.  prvQueueReceiveTask() waits for data to arrive on the queue.
 * When data is received, the task checks the value of the data, then outputs a
 * message to indicate if the data came from the queue send task or the queue
 * send software timer.
 *
 * Expected Behaviour:
 * - The queue send task writes to the queue every 200ms, so every 200ms the
 *   queue receive task will output a message indicating that data was received
 *   on the queue from the queue send task.
 * - The queue send software timer has a period of two seconds, and is reset
 *   each time a key is pressed.  So if two seconds expire without a key being
 *   pressed then the queue receive task will output a message indicating that
 *   data was received on the queue from the queue send software timer.
 *
 * NOTE:  Console input and output relies on Windows system calls, which can
 * interfere with the execution of the FreeRTOS Windows port.  This demo only
 * uses Windows system call occasionally.  Heavier use of Windows system
calls
 * can crash the port.
 */

/* Standard includes. */
#include <stdio.h>
#include <conio.h>

/* Kernel includes. */
#include "FreeRTOS.h"
#include "task.h"
#include "timers.h"
#include "semphr.h"

/* Priorities at which the tasks are created. */
#define mainQUEUE_RECEIVE_TASK_PRIORITY
        ( tskIDLE_PRIORITY + 2 )
```

```
#define        mainQUEUE_SEND_TASK_PRIORITY
        ( tskIDLE_PRIORITY + 1 )

/* The rate at which data is sent to the queue.  The times are converted from
milliseconds to ticks using the pdMS_TO_TICKS() macro. */
#define mainTASK_SEND_FREQUENCY_MS
        pdMS_TO_TICKS( 200UL )
#define mainTIMER_SEND_FREQUENCY_MS
        pdMS_TO_TICKS( 2000UL )

/* The number of items the queue can hold at once. */
#define mainQUEUE_LENGTH                                        ( 2 )

/* The values sent to the queue receive task from the queue send task and the
queue send software timer respectively. */
#define mainVALUE_SENT_FROM_TASK                        ( 100UL )
#define mainVALUE_SENT_FROM_TIMER                       ( 200UL )

/* This demo allows for users to perform actions with the keyboard. */
#define mainNO_KEY_PRESS_VALUE              ( -1 )
#define mainRESET_TIMER_KEY                 ( 'r' )

/*-----------------------------------------------------------*/

/*
 * The tasks as described in the comments at the top of this file.
 */
static void prvQueueReceiveTask( void *pvParameters );
static void prvQueueSendTask( void *pvParameters );

/*
 * The callback function executed when the software timer expires.
 */
static void prvQueueSendTimerCallback( TimerHandle_t xTimerHandle );

/*-----------------------------------------------------------*/

/* The queue used by both tasks. */
static QueueHandle_t xQueue = NULL;
```

```
/* A software timer that is started from the tick hook. */
static TimerHandle_t xTimer = NULL;

/*-----------------------------------------------------------*/

/*** SEE THE COMMENTS AT THE TOP OF THIS FILE ***/
void main_blinky( void )
{
const TickType_t xTimerPeriod = mainTIMER_SEND_FREQUENCY_MS;

   printf( "\r\nStarting the blinky demo. Press \'%c\' to reset the software timer
used in this demo.\r\n\r\n", mainRESET_TIMER_KEY );

        /* Create the queue. */
        xQueue = xQueueCreate( mainQUEUE_LENGTH, sizeof( uint32_t ) );

        if( xQueue != NULL )
        {
                /* Start the two tasks as described in the comments at the top of
this
                file. */
                xTaskCreate( prvQueueReceiveTask,                /* The function
that implements the task. */
                                "Rx",                                   /*
The text name assigned to the task - for debug only as it is not used by the
kernel. */
                                configMINIMAL_STACK_SIZE,               /*
The size of the stack to allocate to the task. */
                                NULL,
        /* The parameter passed to the task - not used in this simple case. */
                                mainQUEUE_RECEIVE_TASK_PRIORITY,/*
The priority assigned to the task. */
                                NULL );
        /* The task handle is not required, so NULL is passed. */

                xTaskCreate( prvQueueSendTask, "TX",
configMINIMAL_STACK_SIZE, NULL,
mainQUEUE_SEND_TASK_PRIORITY, NULL );

                /* Create the software timer, but don't start it yet. */
```

```
		xTimer = xTimerCreate( "Timer",		/* The
text name assigned to the software timer - for debug only as it is not used by the
kernel. */
							xTimerPeriod,		/*
The period of the software timer in ticks. */
							pdTRUE,
/* xAutoReload is set to pdTRUE, so this timer goes off periodically with a
period of xTimerPeriod ticks. */
							NULL,
	/* The timer's ID is not used. */

	prvQueueSendTimerCallback );/* The function executed when the timer
expires. */

		xTimerStart( xTimer, 0 ); /* The scheduler has not started so use a
block time of 0. */

		/* Start the tasks and timer running. */
		vTaskStartScheduler();
	}

	/* If all is well, the scheduler will now be running, and the following
	line will never be reached.  If the following line does execute, then
	there was insufficient FreeRTOS heap memory available for the idle
and/or
	timer tasks   to be created.  See the memory management section on the
	FreeRTOS web site for more details. */
	for( ;; );
}
/*-----------------------------------------------------------*/

static void prvQueueSendTask( void *pvParameters )
{
TickType_t xNextWakeTime;
const TickType_t xBlockTime = mainTASK_SEND_FREQUENCY_MS;
const uint32_t ulValueToSend = mainVALUE_SENT_FROM_TASK;

	/* Prevent the compiler warning about the unused parameter. */
	( void ) pvParameters;
```

```
	/* Initialise xNextWakeTime - this only needs to be done once. */
	xNextWakeTime = xTaskGetTickCount();

	for( ;; )
	{
		/* Place this task in the blocked state until it is time to run again.
		The block time is specified in ticks, pdMS_TO_TICKS() was used
to
		convert a time specified in milliseconds into a time specified in
ticks.
		While in the Blocked state this task will not consume any CPU
time. */
		vTaskDelayUntil( &xNextWakeTime, xBlockTime );

		/* Send to the queue - causing the queue receive task to unblock
and
		write to the console.  0 is used as the block time so the send
operation
		will not block - it shouldn't need to block as the queue should
always
		have at least one space at this point in the code. */
	if (xQueueSend(xQueue, &ulValueToSend, 0U) == pdPASS)
	{
	   printf("Value %d sent into the queue by the task.\t\tIdle
Time %llu%%\r\n\n", ulValueToSend, ulTaskGetIdleRunTimePercent());
	}
	else
	{
	   printf("Task has failed to send value %d to the queue.\n",
ulValueToSend);
	}
	  }
}
/*-----------------------------------------------------------*/

static void prvQueueSendTimerCallback( TimerHandle_t xTimerHandle )
{
const uint32_t ulValueToSend = mainVALUE_SENT_FROM_TIMER;

	/* This is the software timer callback function.  The software timer has a
```

```
	period of two seconds and is reset each time a key is pressed.  This
	callback function will execute if the timer expires, which will only
happen
	if a key is not pressed for two seconds. */

	/* Avoid compiler warnings resulting from the unused parameter. */
	( void ) xTimerHandle;

	/* Send to the queue - causing the queue receive task to unblock and
	write out a message.  This function is called from the timer/daemon task,
so
	must not block.  Hence the block time is set to 0. */
   if (xQueueSend(xQueue, &ulValueToSend, 0U) == pdPASS)
   {
      printf("Value %d sent into the queue by the timer.\t\tIdle
Time %llu%%\r\n", ulValueToSend, ulTaskGetIdleRunTimePercent());
   }
   else {
      printf("Software timer has failed to send value %d to the queue.\n",
ulValueToSend);
   }
}
/*-----------------------------------------------------------*/

static void prvQueueReceiveTask( void *pvParameters )
{
uint32_t ulReceivedValue;

	/* Prevent the compiler warning about the unused parameter. */
	( void ) pvParameters;

	for( ;; )
	{
		/* Wait until something arrives in the queue - this task will block
		indefinitely provided INCLUDE_vTaskSuspend is set to 1 in
		FreeRTOSConfig.h.  It will not use any CPU time while it is in the
		Blocked state. */
		xQueueReceive( xQueue, &ulReceivedValue,
portMAX_DELAY );
```

```
        /* Enter critical section to use printf. Not doing this could potentially cause
           a deadlock if the FreeRTOS simulator switches contexts and another task
           tries to call printf - it should be noted that use of printf within
           the FreeRTOS simulator is unsafe, but used here for simplicity. */
        taskENTER_CRITICAL();
        {
            /*  To get here something must have been received from the queue, but
            is it an expected value?  Normally calling printf() from a task is not
            a good idea.  Here there is lots of stack space and only one task is
            using console IO so it is ok.  However, note the comments at the top of
            this file about the risks of making Windows system calls (such as
            console output) from a FreeRTOS task. */
            if (ulReceivedValue == mainVALUE_SENT_FROM_TASK)
            {
                printf("Value %d read from the queue by the task.\t\tIdle
Time %llu%%\r\n", ulReceivedValue, ulTaskGetIdleRunTimePercent());
            }
            else if (ulReceivedValue == mainVALUE_SENT_FROM_TIMER)
            {
                printf("Value %d read from the queue by the task.\t\tIdle
Time %llu%%\r\n\n", ulReceivedValue, ulTaskGetIdleRunTimePercent());
            }
            else
            {
                printf("Unexpected message\r\n");
            }
        }
        taskEXIT_CRITICAL();
          }
}
/*-----------------------------------------------------------*/

/* Called from prvKeyboardInterruptSimulatorTask(), which is defined in
main.c. */
void vBlinkyKeyboardInterruptHandler( int xKeyPressed )
{
    /* Handle keyboard input. */
    switch ( xKeyPressed )
    {
    case mainRESET_TIMER_KEY:
```

```
        if ( xTimer != NULL )
        {
            /* Critical section around printf to prevent a deadlock
               on context switch. */
            taskENTER_CRITICAL();
            {
                printf("\r\nResetting software timer.\r\n\r\n");
            }
            taskEXIT_CRITICAL();

            /* Reset the software timer. */
            xTimerReset( xTimer, portMAX_DELAY );
        }

        break;

    default:
        break;
    }
}
```

OUTPUT

```
Trace started.
The trace will be dumped to the file "Trace.dump" whenever a call to configASSERT()
fails or the 't' key is pressed.
Note that the trace output uses the ring buffer mode, meaning that the output trace
will only be the most recent data able to fit within the trace recorder buffer.

Starting the blinky demo. Press 'r' to reset the software timer used in this demo.

Value 100 read from the queue by the task.          Idle Time 96%
Value 100 sent into the queue by the task.          Idle Time 96%

Value 100 read from the queue by the task.          Idle Time 97%
Value 100 sent into the queue by the task.          Idle Time 97%

Value 100 read from the queue by the task.          Idle Time 97%
Value 100 sent into the queue by the task.          Idle Time 97%

Value 100 read from the queue by the task.          Idle Time 98%
Value 100 sent into the queue by the task.          Idle Time 98%

Value 100 read from the queue by the task.          Idle Time 98%
Value 100 sent into the queue by the task.          Idle Time 98%

Value 100 read from the queue by the task.          Idle Time 98%
Value 100 sent into the queue by the task.          Idle Time 98%

Value 100 read from the queue by the task.          Idle Time 98%
Value 100 sent into the queue by the task.          Idle Time 98%

Value 100 read from the queue by the task.          Idle Time 98%
Value 100 sent into the queue by the task.          Idle Time 98%

Value 100 read from the queue by the task.          Idle Time 98%
Value 100 sent into the queue by the task.          Idle Time 98%

Value 200 sent into the queue by the timer.         Idle Time 98%
Value 200 read from the queue by the task.          Idle Time 98%

Value 100 read from the queue by the task.          Idle Time 98%
Value 100 sent into the queue by the task.          Idle Time 98%
```

References

FreeRTOS Installation	https://www.freertos.org/a00104.html
Win32 Solution	https://www.freertos.org/simple-freertos-demos.html
FreeRTOS Documentation	https://www.freertos.org/Documentation/Mastering-the-FreeRTOS-Real-Time-Kernel.v1.0.pdf